

Practice Assignment

API Fundamentals & Postman Mastery

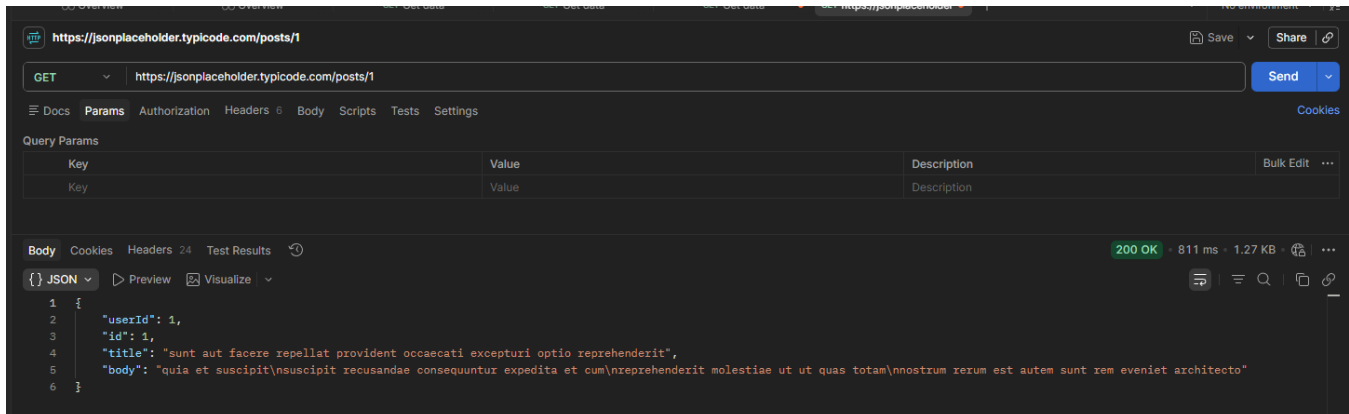
Api for every task: <https://jsonplaceholder.typicode.com>

Tasks

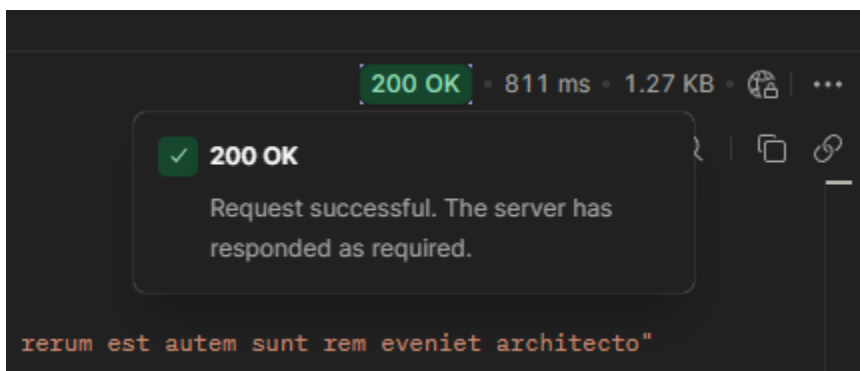
TASK 1 Reading Data Without Headers

- Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.
- Record the status code you get back.
- In one sentence, explain why this request works even with nothing in the Headers tab.

Task 1(a): Send a GET Request



Task 1(b): Record the Status Code : Status Code: 200 OK



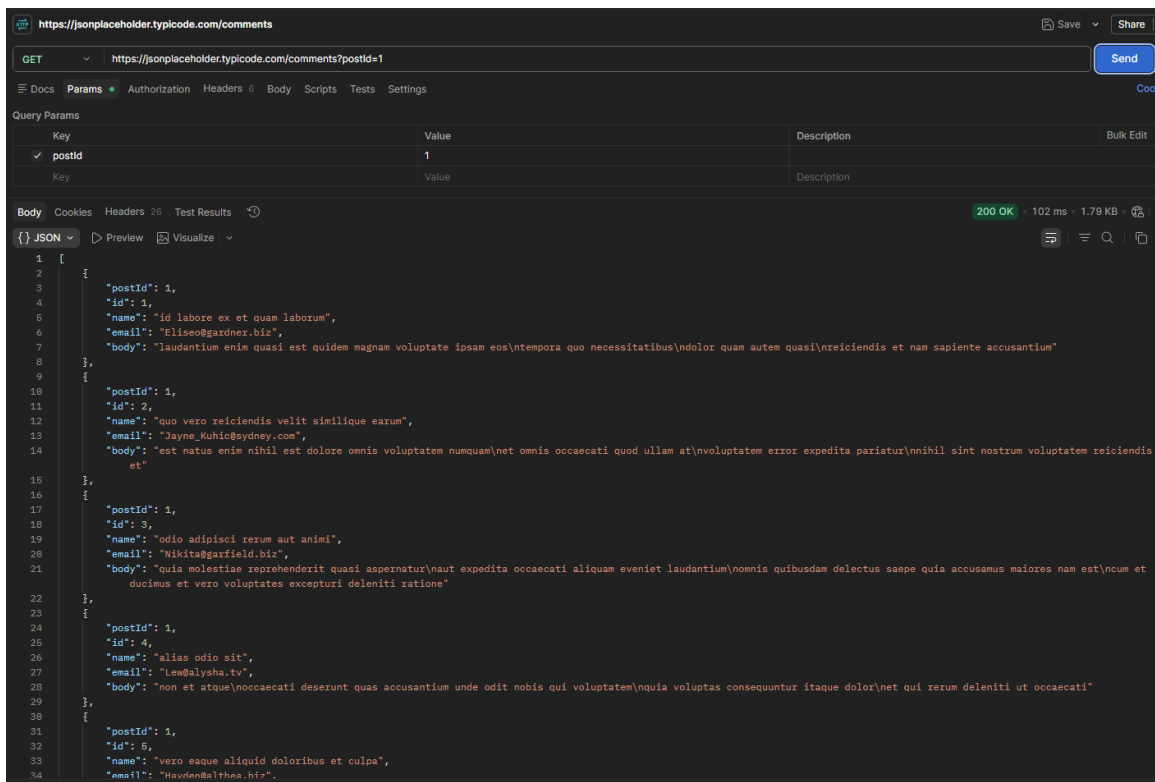
Task 1(c): Written Answer

The request works without any headers because it is a public GET request. The JSONPlaceholder API does not require authentication or special headers to retrieve public data.

TASK 2 Filtering With Query Parameters

- Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.
- Confirm the URL automatically updates to include ?postId=1.
- Now combine two filters at once on /todos: userId = 1 and completed = false.
- In one sentence, explain what changed in the results compared to not using any params.

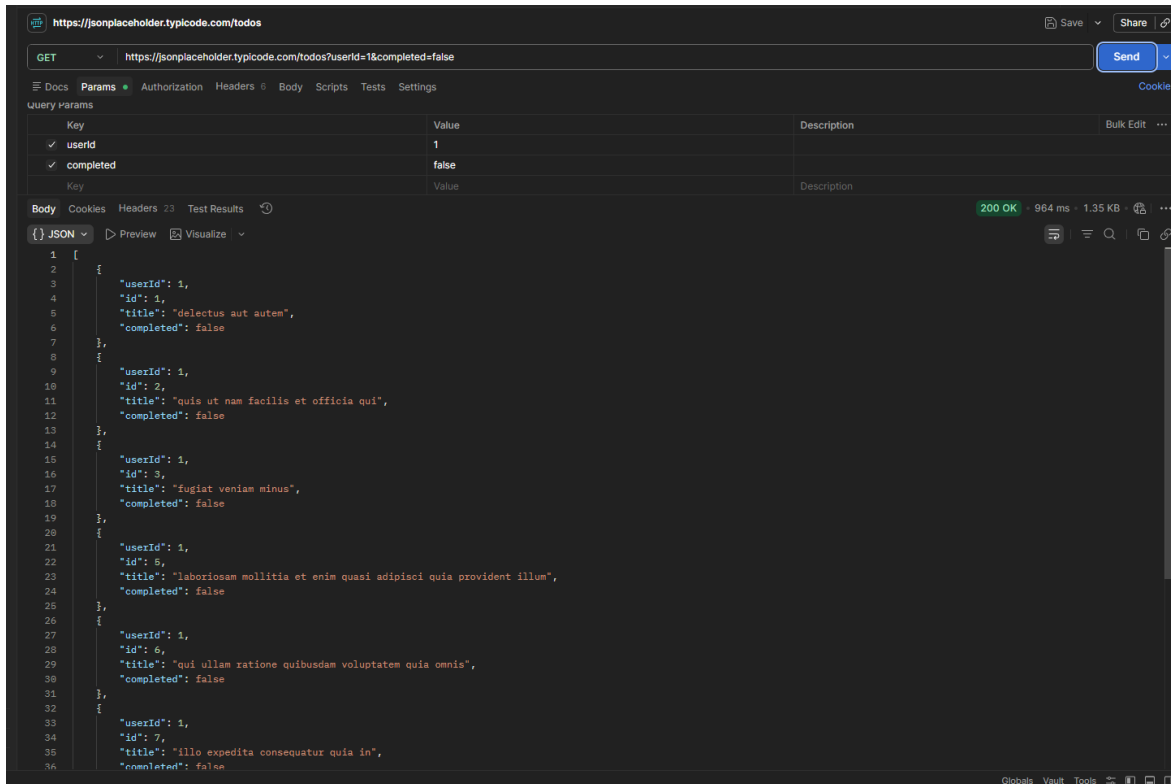
Task 2(a): Filter Comments by postId



Task 2(b): Confirm the URL

Yes, Postman automatically updated the URL to include `?postId=1` when the query parameter was added.

Task 2(c): Use Two Query Parameters



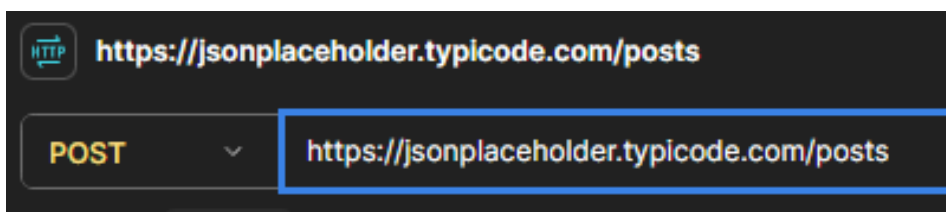
Task 2(d): Written Answer

Using query parameters filters the results returned by the API. Instead of receiving all records, the server returns only the data that matches the specified conditions, making the response smaller and more relevant.

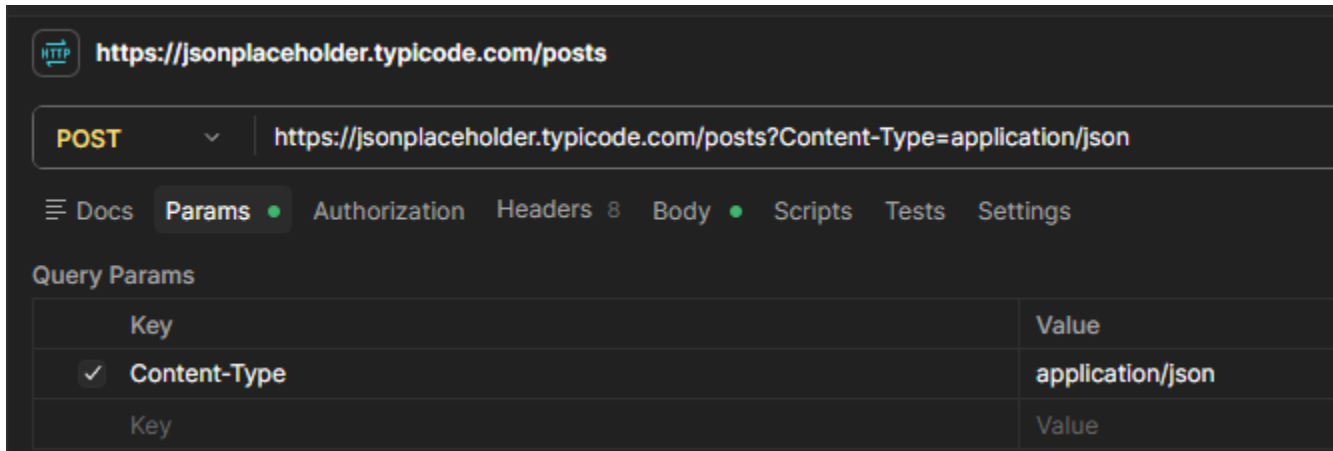
TASK 3 Sending Data With Headers & a Body

- Send a POST request to `/posts`.
- In the Headers tab, add: `Content-Type: application/json`
- In the Body tab (raw, JSON), send a new post with a title, body, and `userId` of your choice.
- Record the status code and the new id the server assigned.

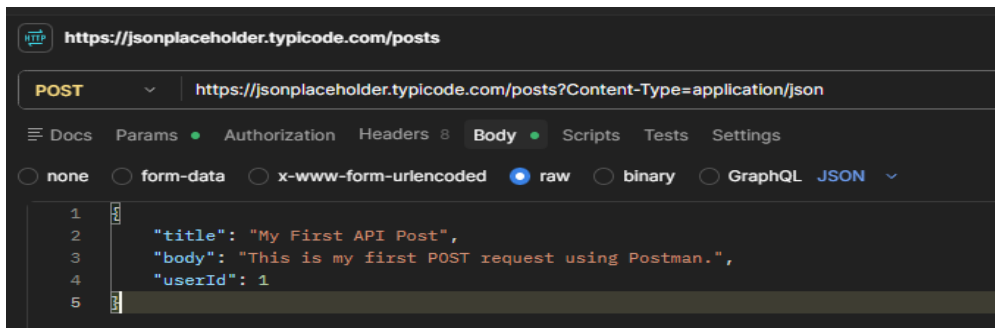
Task 3(a): Create a POST Request



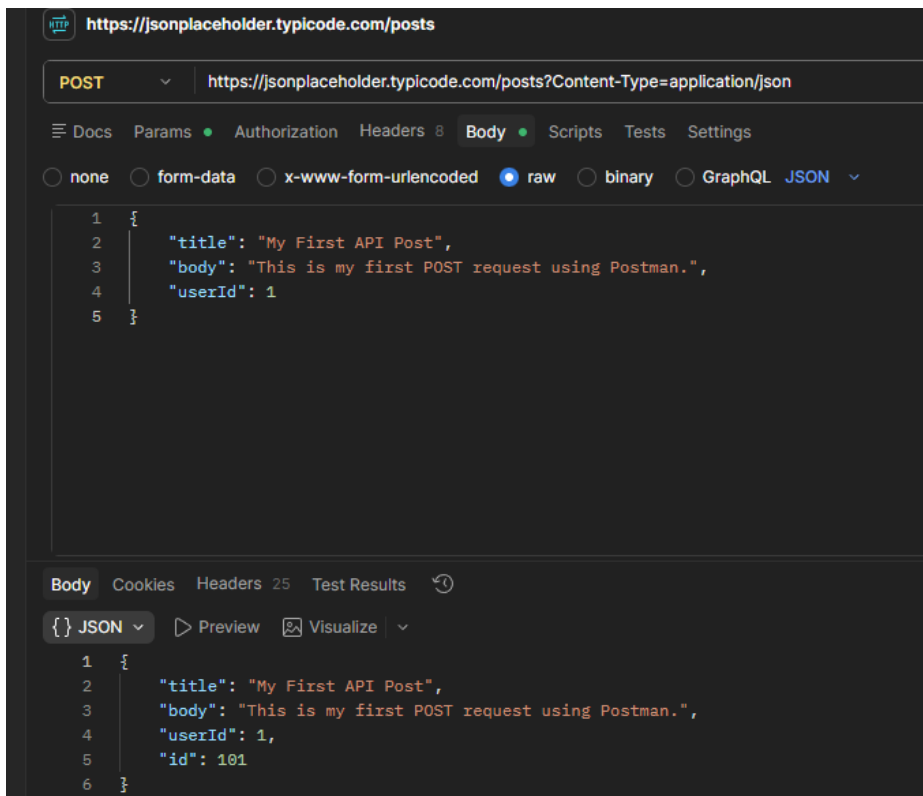
Task 3(b): Add the Header



Task 3(c): Add the Request Body



Task 3(d): Record the Status Code



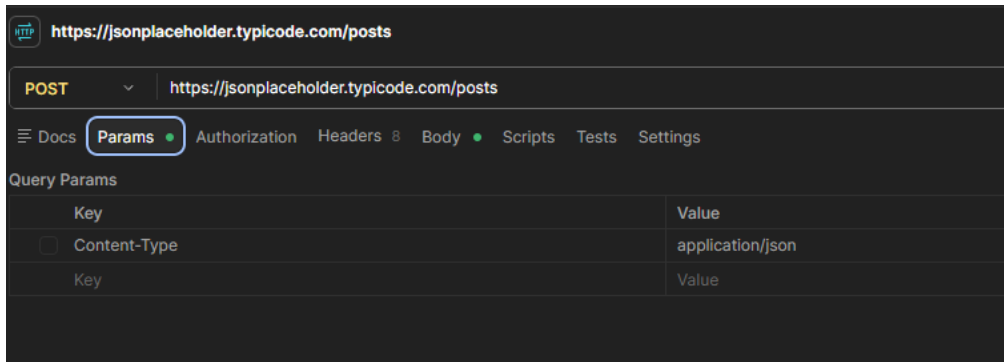
- **Status Code:** 201 Created
- **New ID Assigned:** 101

A POST request is used to send new data to the server. The Content-Type: application/json header tells the server that the request body contains JSON data. The server responds with 201 Created and returns the newly created resource with an assigned ID.

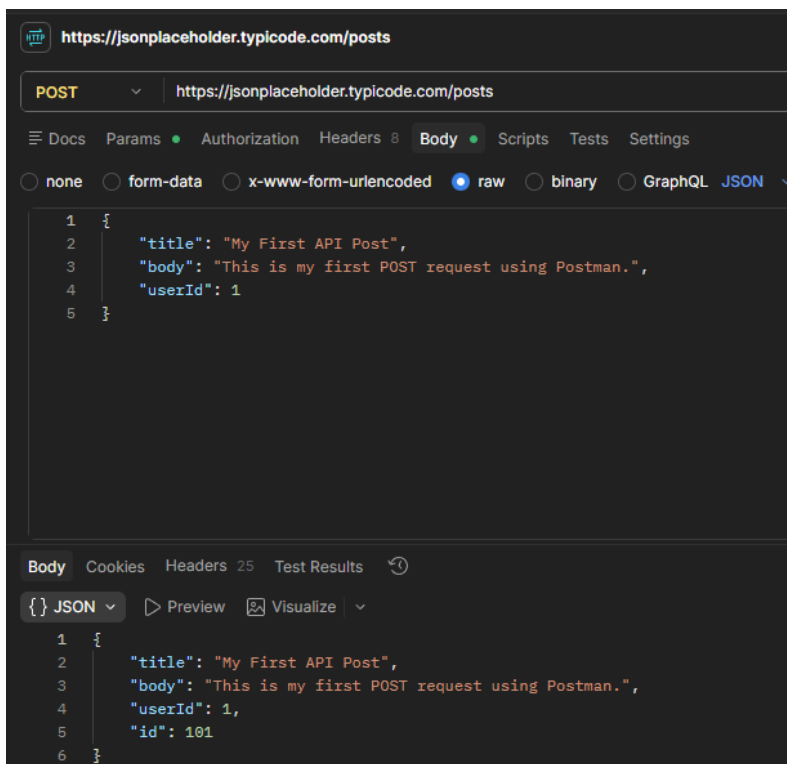
TASK 4 Headers vs. No Headers

- Repeat Task 3's POST request, but this time remove the Content-Type header before sending.
- Compare the response to Task 3 — is it different? Record what you observe.
- In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

Task 4(a): Send POST Request Without the Header



Task 4(b): Compare the Response



With Content-Type

Status: 201 Created

Request processed

Correctly identifies JSON data JSONPlaceholder still accepts it

Without Content-Type

Status: 201 Created

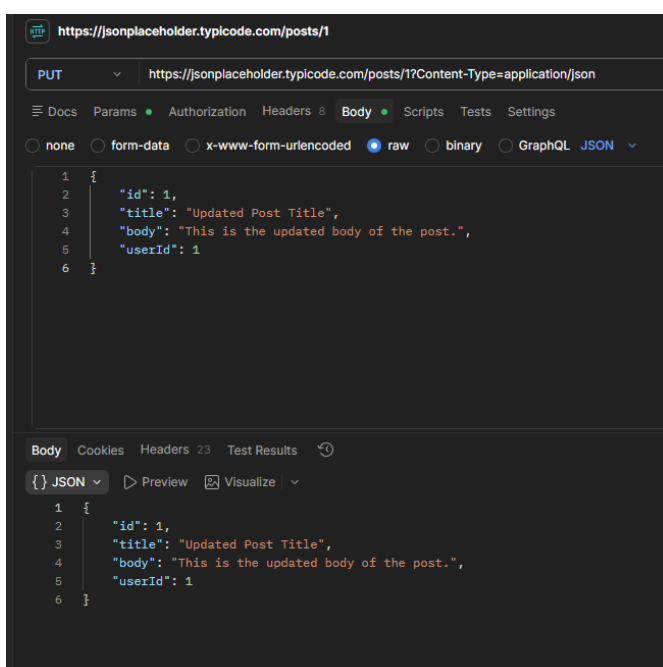
Request still processed

Task 4(c): Written Answer

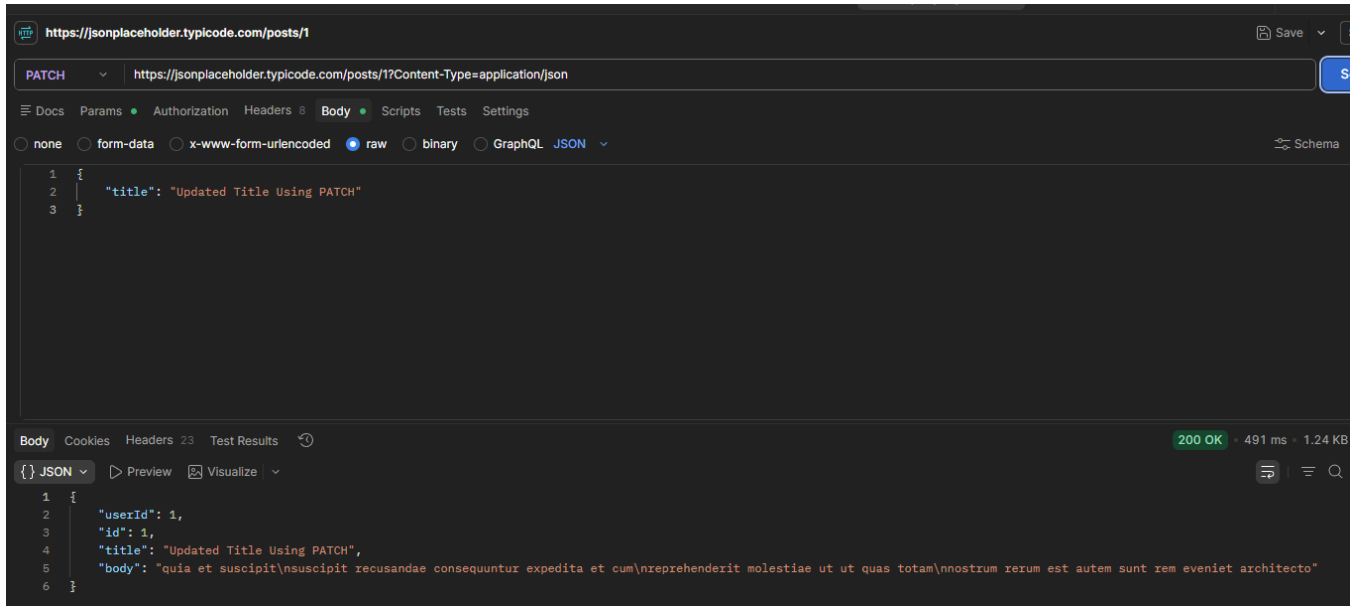
Headers are important because they tell the server how to interpret the request data. The Content-Type: application/json header informs the server that the request body contains JSON. Although JSONPlaceholder accepts the request without this header, a real API may reject the request or return an error such as 400 Bad Request or 415 Unsupported Media Type if the correct header is missing.

TASK 5 PUT vs. PATCH

- Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).
- Send a PATCH request to the same /posts/1 that changes only the title.
- In one sentence, explain the difference between what PUT sent and what PATCH sent.

Task 5(a): Send a PUT Request

Task 5(b): Send a PATCH Request



Task 5(c): Written Answer

PUT replaces the entire resource, so the request body must contain all fields of the resource. PATCH updates only the specified fields, so the request body can contain only the fields that need to be changed.

PUT

Replaces the entire resource

Send all fields (id, title, body, userId)

Used for complete updates

PATCH

Updates only selected fields

Send only the fields to change

Used for partial updates

Final Challenge — A Realistic Request Flow

TASK 6 The Mini Feedback System

Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

- a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.
- b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.
- c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.
- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.
- e) Record the status code for every step above (a–d) in a short table or list.
- f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

Step (a): Read Existing Comments

The screenshot shows a REST client interface with the following details:

- URL:** `https://jsonplaceholder.typicode.com/comments`
- Method:** GET
- Query Params:** `https://jsonplaceholder.typicode.com/comments?postId=1`
- Status:** 200 OK
- Response Body (JSON):**

```

1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2,
12    "name": "quo vero reiciendis velit similique earum",
13    "email": "Jayne_Kuhic@sydney.com",
14    "body": "\"est natus enim nihil est dolore omnis voluptatem numquam\nnet omnis occaecati quod ullam at\nvoluptatem error expedita pariatur\nnnihil sint nostrum voluptatem reiciendis et\""
15  },
16  {
17    "postId": 1,
18    "id": 3,
19    "name": "odio adipisci rerum aut animi",
20    "email": "Nikita@garfield.biz",
21    "body": "\"quia molestiae reprehenderit quasi aspernatur\naut expedita occaecati aliquam eveniet laudantium\nnomnis quibusdam delectus saepe quia accusamus maiores nam est\ncum et ducimus et vero voluptates excepturi deleniti ratione\""
22  },
23 ]

```


Step (b): Add a New Comment (No Headers)

The screenshot shows a REST client interface with the following details:

- URL:** `https://jsonplaceholder.typicode.com/comments`
- Method:** `POST`
- Body Type:** `raw` (selected)
- Request Body (JSON):**

```
1 {
2   "name": "Amarsinh Gohil",
3   "email": "amarsinh@example.com",
4   "body": "This is my feedback."
5 }
```
- Response:** `201 Created`
- Response Body (JSON):**

```
1 {
2   "name": "Amarsinh Gohil",
3   "email": "amarsinh@example.com",
4   "body": "This is my feedback.",
5   "id": 501
6 }
```

Step (c): Add the Content-Type Header

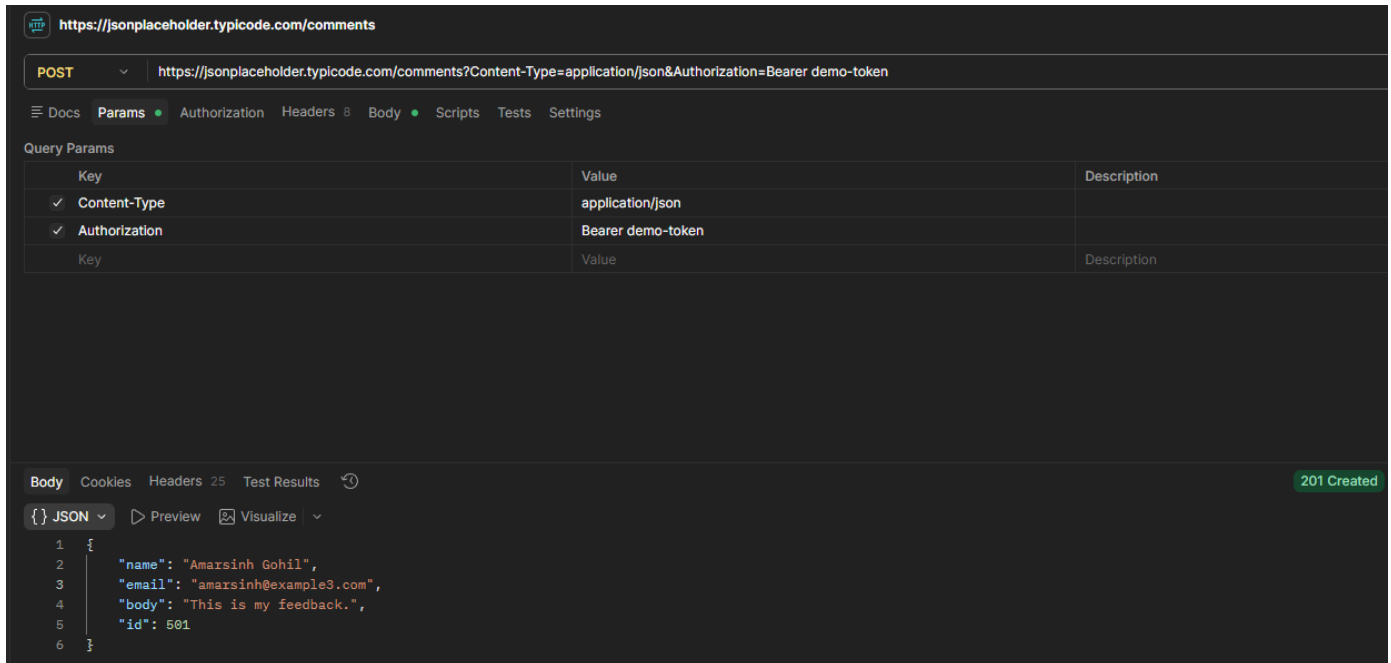
The screenshot shows a REST client interface with the following details:

- URL:** `https://jsonplaceholder.typicode.com/comments?Content-Type=application/json`
- Method:** `POST`
- Body Type:** `raw` (selected)
- Request Body (JSON):**

```
1 {
2   "name": "Amarsinh Gohil",
3   "email": "amarsinh@example2.com",
4   "body": "This is my feedback."
5 }
```
- Response:** `201 Created`
- Response Body (JSON):**

```
1 {
2   "name": "Amarsinh Gohil",
3   "email": "amarsinh@example2.com",
4   "body": "This is my feedback.",
5   "id": 501
6 }
```

Step (d): Add an Authorization Header



Step (e): Record the Status Codes

Step	Request	Status Code
(a)	GET /comments?postId=1	200 OK
(b)	POST /comments (No Headers)	201 Created
(c)	POST /comments (Content-Type)	201 Created
(d)	POST /comments (Content-Type + Authorization)	201 Created

Step (f): Written Answer

If this were a real API, the Authorization header would contain a valid authentication token. The server would verify the token before processing the request. If the token was missing or invalid, the server would reject the request with an error such as 401 Unauthorized or 403 Forbidden.